INSTRUCTOR-LED · HANDS-ON · FIVE DAYS

Agentic AI Engineering

A five-day programme that takes working software engineers from their first model call to a production-shaped AI agent — built, measured, hardened, and understood well enough to own after the trainer leaves.

DURATION	HANDS-ON	DELIVERY	DEFAULT STACK
5 days · 1,950 min	**~75% of contact time**	**On-site or virtual**	**LangGraph / LangChain**

What each engineer can do afterwards

Not "has been introduced to" — can do, unaided, on the team's own systems.

- ✓ Write a tool-using agent from first principles, and explain exactly what a framework adds on top.
- ✓ Build a retrieval pipeline and **state its score against a golden set** rather than asserting it works.
- ✓ Run an evaluation suite in CI that fails the build on a quality regression.
- ✓ Find and close a prompt-injection path in their own build.
- ✓ Get typed, validated, schema-conformant output from a model and handle it when it fails.
- ✓ Model an agent as a graph with branching, durable state and human approval gates.
- ✓ Trace a run, attribute its cost, and reduce latency with evidence.
- ✓ Name the failure modes of a planned system and the guardrail for each.

WHAT EACH PARTICIPANT TAKES AWAY

- A **working agent** they wrote themselves, without a framework, and the framework version alongside it.
- A **retrieval pipeline with a measured score** and the golden set it was measured against.
- An **evaluation suite** wired into a build, that catches a regression.
- A traced, cost-attributed run they have already optimised.
- A written course manual and reference implementation to rebuild everything independently.

WHAT THE ORGANISATION GAINS

- **In-house capability that stays** — engineers who can build and maintain agent systems without external help.
- A shared vocabulary between engineering, architecture and security for reviewing AI work.
- The ability to **evaluate AI quality with numbers**, which is what makes AI work reviewable and fundable.
- Reduced dependence on vendors for systems the team could build and own.
- A team that can tell a promising AI proposal from an unworkable one.

WHO THIS IS FOR

Designed for **working software engineers** who write code day to day — backend, full-stack, platform or data. Python is the working language; engineers from Java, C#/.NET or similar backgrounds are well served, and a short primer is provided in advance so everyone starts level.

It suits teams who have used AI tools and now need to **build with them properly** — reliably, measurably, and in production. Architects and technical leads take the same programme and get the review vocabulary that comes with it.

ONE SYSTEM, GROWN ACROSS FIVE DAYS

Rather than a new toy example per topic, the programme builds a single system that deepens each day: an internal support agent that answers from the organisation's own documents, checks live system state through tools, decides, acts — and is then evaluated, guarded and traced until it is fit to put in front of people. The domain is set to the client's own during scoping.

Day by day

Timed to an eight-hour day with 90 minutes of breaks — 390 minutes of instruction per day. The extra day over a compressed four-day version goes into the foundations and the hand-built agent, where a participant either becomes solid or spends the rest of the week catching up.

DAY 1 The model as a component

75 + 120 + 120 + 75 = 390 min

> Treating the model as an unreliable function with a useful contract — which is the mental model everything else rests on.

75 MIN LAB

How the model behaves

Context, token budgets, temperature and non-determinism — established by running things and watching, rather than by slides.

LAB

Run the same prompt repeatedly and vary the parameters. Find the setting at which it stops being repeatable, and reason about why.

120 MIN LAB

Structured output

The highest-value skill for a backend engineer: a typed, validated object instead of prose. Schemas, JSON mode, validation, and recovery when parsing fails.

LAB

Turn free text into a validated object. Then break it deliberately — bad enum, missing field, truncated response — and handle each case.

120 MIN LAB

Tool calling — the core primitive

How a tool gets chosen, how arguments arrive, and why tool-schema design is really API design. Validating arguments before anything executes.

LAB

Expose three tools over a stub internal API and watch the model choose. Then make it choose wrong on purpose, and see what that costs.

75 MIN CONSOLIDATION

Supported build and catch-up

Open working time with the instructor circulating. Anyone behind gets individual attention; anyone ahead extends their build.

> SESSION
> Everyone finishes Day 1 with the same working baseline, which is what makes Day 2 possible for the whole room.

DAY 2 The agent loop, built by hand

135 + 105 + 105 + 45 = 390 min

> A full day writing the loop with no framework. Day 4's framework then reads as a simplification of something already understood, rather than as magic.

135 MIN LAB

Building the loop

Reason, call, observe, repeat. The whole cycle written from first principles, with every decision visible in code the engineer wrote.

> LAB
> A hand-rolled agent that answers a genuinely multi-step question, built from nothing.

105 MIN LAB

Termination, limits and error propagation

Stop conditions, iteration caps, and how errors travel through a loop that calls a model that calls a tool. Where a loop hangs, and where it spirals.

> LAB
> Make your own agent loop fail in three different ways, then make each failure bounded and legible.

105 MIN LAB

Extending it

More tools, harder questions, and the point at which a single prompt stops being enough and structure has to take over.

> LAB
> Add tools until the agent starts choosing badly — then work out whether the fix is a better description, a better schema, or fewer tools.

45 MIN CONSOLIDATION

Review and shared baseline

Comparing implementations across the room. Different people solve the loop differently, and the comparison teaches more than the build did.

DAY 3 Retrieval, end to end

75 + 105 + 135 + 75 = 390 min

> Built around measurement. A team that cannot score its retrieval cannot improve it, and this is the part most programmes leave out.

75 MIN LAB

Retrieval end to end, deliberately mediocre

Load, chunk, embed, store, retrieve, generate — built fast and imperfectly on purpose, so the rest of the day has something real to fix.

> LAB
> Stand up the pipeline on a supplied corpus, ask ten questions, and collect the bad answers. They are the day's material.

105 MIN LAB

Why it failed — chunking and representation

Fixed, recursive, semantic and structure-aware splitting. Chunk size and overlap. Metadata worth carrying. Why tables and scanned pages break everything.

> LAB
> Re-chunk three ways, re-run the same ten questions, and put the differences side by side.

135 MIN LAB

Measuring retrieval

Building a golden set. Hit rate, recall at k, mean reciprocal rank. Hybrid search, reranking and metadata filters — each adopted only if the number moves.

> LAB
> A twenty-question golden set, a baseline, then three interventions each scored against it. The team leaves with a number they can defend to their own architects.

75 MIN LAB

Grounding, citation and refusal

Prompting for citations, forcing "not in the sources", and handling documents that contradict each other. Then wiring retrieval into Day 2's agent as a tool.

> LAB
> The agent answers from the corpus with citations — and correctly refuses a question the corpus cannot answer.

DAY 4 The framework: state, control, coordination

105 + 90 + 105 + 90 = 390 min

> The one day where the stack matters. Same outcomes and same labs whichever is chosen.

CHOICE OF STACK — ONE, SELECTED AT BOOKING

LangGraph + LangChain · DEFAULT

- Graph nodes, edges, typed state schema
- Conditional edges, cycles, bounded retries
- Checkpointers, thread state, interrupt and resume
- Supervisor, router and plan-execute patterns

AWS Bedrock

- Bedrock Agents / AgentCore, session state
- Action groups backed by Lambda
- Knowledge Bases alongside the Day 3 pipeline
- Step Functions for multi-agent orchestration

105 MIN LAB

Port the hand-rolled loop onto the framework

The same agent from Day 2, rebuilt as a graph. Because the loop was written by hand, every abstraction now has something concrete underneath it.

> LAB
>
> Same behaviour, a fraction of the code. Then diff the two and name exactly what the framework bought.

---

90 MIN LAB

Branching, routing and recovery

Conditional paths on state, fallbacks, bounded retries, and cycles that terminate. Where a router beats a bigger prompt.

> LAB
>
> Add a routing node choosing between retrieval, a tool call and answering directly — with a fallback when the first choice fails.

---

105 MIN LAB

Persistence and the human in the loop

Durable state across turns, resuming an interrupted run, and approval gates in front of anything destructive — the pattern that makes agents acceptable to a risk function.

> LAB
>
> The agent pauses for explicit human approval before writing to a system, then survives a process restart mid-run.

---

90 MIN LAB

Multi-agent patterns, and MCP

Supervisor, router, plan-and-execute — with the honest counter-case that most problems sold as multi-agent are one agent with better tools. Then MCP, for exposing an existing internal service to any agent.

> LAB
>
> A two-agent handoff plus one internal service wrapped as an MCP tool and called from the graph.

DAY 5 Making it survive production

105 + 90 + 90 + 105 = 390 min

> The day that decides whether anything built this week reaches real users.

105 MIN LAB

Evaluation and the regression suite

Assertion-based checks against model-as-judge, and the specific ways a judge misleads you. Evaluating retrieval and generation separately. Running it in CI.

> **LAB**
> Build an eval suite over the week's system, wire it to the build, then break something and watch the build go red.

90 MIN LAB

Guardrails, verifiers and blast radius

Input and output validation, verifier patterns that re-check a claim before it is surfaced, prompt injection arriving through retrieved documents and tool results, allowlists, and capping what a bad run can touch.

> **LAB**
> Attack your own agent through a poisoned source document, close the hole, and prove the fix with the eval suite.

90 MIN LAB

Tracing, cost and latency

What to log and what never to log. Token accounting per request, per user, per feature. Exact and semantic caching, model routing, streaming, timeouts.

> **LAB**
> Trace the slowest run, then cut its cost and latency measurably without moving the eval score.

105 MIN REVIEW

Deployment, failure modes, and your own system

Packaging, statefulness, concurrency, rollout, and model-version changes that shift behaviour underneath you. How agent incidents differ from service incidents — they degrade quietly rather than page anyone. The room then turns to its own use cases.

> **CLOSING SESSION**
> Each team presents the system it intends to build, with a live architecture review, named risks and a first evaluation plan.

How it is delivered

THE TEACHING DAY

Eight hours with 90 minutes of breaks — **390 minutes of instruction**. Every day is timed to that budget and the blocks sum to it.

GROUP SIZE

Hands-on work runs in **groups of five to six**, with a second facilitator for larger cohorts, so labs stay practical rather than becoming a demonstration.

VIRTUAL DELIVERY

Delivered as **ten half-days across three to four weeks** — same content and same labs, with better attendance and retention than consecutive full days.

COMPRESSED OPTION

The programme **compresses to four days** where scheduling requires it, by tightening the consolidation sessions and shortening supported build time.

LANGUAGE AND PACING

Delivered in English, with pacing and written materials adjusted for cohorts working in a second language.

GETTING READY

A short technical readiness call and a setup checklist are provided ahead of delivery, so the first morning starts with teaching rather than configuration.

Your trainer

Arjun Thakur

Principal AI Engineer · AI Architect · Corporate AI Trainer

- **15+ years of production software engineering** across payments, travel, fintech and education technology — spanning senior engineering, engineering management and director-level roles at global product companies.
- **Builds and operates a production agentic AI product.** Agent orchestration, retrieval, evaluation and guardrails running live with paying customers — this material is taught by someone who runs it, not only by someone who has read about it.
- **Deep backend and distributed-systems background** — Python, Java, microservices, event-driven architecture, PostgreSQL and cloud infrastructure, which is what makes the production half of this programme credible.
- **AWS Certified Solutions Architect.** M.Tech in Computer Science and Engineering.
- **Experienced with enterprise engineering cohorts**, including multi-day hands-on programmes for mixed groups of developers, QA, DevOps and architects, delivered at around 80% hands-on with capstone projects.

EMAIL
arjun@arjunthakur.dev

WEBSITE
arjunthakur.dev

LINKEDIN
linkedin.com/in/thakurarjun247